

Artificial Intelligence Assignment 4

Travelling Salesman Problem Using Greedy Technique and Simulated Annealing

Student ID: 2205029

September 8, 2026

1 Introduction

The Travelling Salesman Problem (TSP) is a classical combinatorial optimization problem in which a salesman must visit every city in a given set exactly once and return to the starting city, while minimizing the total travelling cost. Since TSP is NP-hard, exact algorithms become computationally infeasible as the number of cities N grows, motivating the use of heuristic and metaheuristic approaches.

This report documents the implementation, testing, and experimental evaluation of two approaches to TSP:

1. **Greedy Technique (Nearest Neighbor)** — a deterministic, constructive heuristic that builds a tour by always moving to the cheapest unvisited city.
2. **Simulated Annealing (SA)** — a probabilistic metaheuristic inspired by the physical annealing process, capable of escaping local optima by occasionally accepting worse solutions.

Both techniques were implemented in Python, tested on five TSP instances of increasing size (10, 20, 50, 100, and 200 cities), and compared using execution time, tour cost, and solution quality. In addition, the effect of Simulated Annealing's key parameters — initial temperature T_0 and cooling rate α — was studied, along with the effect of the initial solution (random vs. greedy) on final solution quality.

2 Problem Description

Given N cities and an $N \times N$ travelling-cost matrix C , where $C[i][j]$ is the cost of travelling from city i to city j , the objective is to find a tour that:

- starts from City 0,
- visits every city exactly once,
- returns to City 0,
- minimizes the total travelling cost.

2.1 Assumptions

- $2 \leq N \leq 500$
- $C[i][i] = 0$ for all i
- All other costs are strictly positive
- The cost matrix may be symmetric or asymmetric
- City numbering starts from 0, and every tour begins at City 0

2.2 Input Format

The input file begins with an integer N , followed by N lines each containing N space-separated integers representing one row of the cost matrix:

```
N
c[0][0] c[0][1] ... c[0][N-1]
c[1][0] c[1][1] ... c[1][N-1]
...
c[N-1][0] ... c[N-1][N-1]
```

3 Implementation

The solution is implemented as a single Python class, `TSP`, which encapsulates the cost matrix and all algorithmic operations. Two driver scripts were used:

- `2205029.py` — runs the Greedy algorithm once, and Simulated Annealing (greedy-initialized) across a full 3×3 grid of $T_0 \in \{100, 1000, 5000\}$ and $\alpha \in \{0.9, 0.95, 0.995\}$, with a fixed random seed of **42**. This single run served jointly as Experiment 1 (Greedy vs. SA), Experiment 2 (effect of T_0), and Experiment 3 (effect of α), since the same grid of runs answers all three questions simultaneously and improves the robustness of the comparison (every T_0/α combination is evaluated under identical, reproducible randomness).
- `2205029exp4.py` — Experiment 4, comparing random vs. greedy-initialized Simulated Annealing using a fixed configuration ($T_0 = 100$, $\alpha = 0.995$), repeated 5 times *without* a fixed seed so that the run-to-run variance of SA can be observed, on all five test instances.

Per the assignment’s implementation requirements, the following functions/ modules are present: `readInput()`, `calculateTourCost()`, `greedyTSP()`, `generateRandomTour()`, `generateNeighbor()`, `simulatedAnnealing()`, and `printResults()`.

3.1 Reading the Input

`readInput()` opens the file, reads N from the first line, and then reads the $N \times N$ cost matrix row by row into `self.graph`. A boolean `visited` array of size N is also initialized.

Listing 1: Input parsing

```
1 def readInput(self, filename):
2     with open(filename, 'r') as f:
3         self.N = int(f.readline())
```

```

4     self.visited = [False] * self.N
5     self.graph = []
6     for i in range(self.N):
7         row = list(map(int, f.readline().split()))
8         self.graph.append(row)

```

This directly matches the required input specification (Section 3 of the assignment): the first line is N , followed by N rows of the cost matrix.

3.2 Greedy Technique (Nearest Neighbor)

Starting from City 0, the greedy heuristic repeatedly selects the nearest unvisited city, marks it visited, and moves to it, finally returning to City 0.

Listing 2: Greedy Nearest-Neighbor construction

```

1 def greedyTSP(self):
2     self.visited[0] = True
3     start_time = time.time()
4     current = 0
5     path = [0]
6     for i in range(1, self.N):
7         next_city = -1
8         min_cost = math.inf
9         for j in range(self.N):
10            if not self.visited[j] and self.graph[current][j] < min_cost:
11                min_cost = self.graph[current][j]
12                next_city = j
13            path.append(next_city)
14            self.visited[next_city] = True
15            current = next_city
16        self.greedyTSP_time = time.time() - start_time
17        path.append(0) # Return
18    return path

```

The outer loop runs $N - 1$ times, and the inner loop scans up to N cities each time, giving the expected $O(N^2)$ time complexity (analysed further in Section 10).

3.3 Tour Cost Calculation

`calculateTourCost()` traverses the path and sums the edge costs between consecutive cities, also constructing a human-readable "0 -> 1 -> ... -> 0" string for reporting.

Listing 3: Tour cost evaluation

```

1 def calculateTourCost(self, path):
2     cost = 0
3     prev = -1
4     printstring = ""
5     for i in path:
6         if prev != -1:
7             printstring += f"-> {i} "
8             cost += self.graph[prev][i]
9         else:
10            printstring += str(i) + " "
11            prev = i
12    return cost, printstring

```

This is an $O(N)$ operation, since the path has $N + 1$ entries (including the return to City 0).

3.4 Initial Solution for Simulated Annealing

Both required methods from Section 7.3 of the specification are implemented:

- **Random initial solution** — `generateRandomTour()` shuffles cities $1, \dots, N - 1$ and fixes City 0 at both ends.
- **Greedy initial solution** — the tour produced by `greedyTSP()` is reused directly as the SA starting point when `useGreedy=True`.

Listing 4: Random initial tour generation

```
1 def generateRandomTour(self):
2     path = list(range(1, self.N))
3     random.shuffle(path)
4     path = [0] + path + [0]
5     return path
```

3.5 Neighborhood Generation

The assignment’s Section 8.1 specifies a 2-opt / segment-reversal neighborhood operator. This implementation strictly follows that specification.

The `generateNeighbor()` function selects two random interior indices, `index1` and `index2` (where $1 \leq \text{index1} < \text{index2} \leq N - 1$), ensuring that City 0 remains fixed at the first and last positions. It then reverses the segment of the path between these two indices in-place.

Listing 5: Neighbor generation (2-Opt / Segment Reversal)

```
1 def generateNeighbor(self, path):
2     if self.N < 3:
3         return path
4     index1 = random.randint(1, self.N - 2)
5     index2 = random.randint(index1 + 1, self.N - 1)
6     for i in range((index2 - index1 + 1) // 2):
7         path[index1 + i], path[index2 - i] = path[index2 - i], path[index1 + i]
8     return path
```

The `for` loop performs the in-place reversal by swapping elements symmetrically from the outside of the chosen segment moving inward, requiring exactly $\lfloor (\text{index2} - \text{index1} + 1)/2 \rfloor$ swaps.

This operator guarantees that every generated neighbor is a valid TSP tour: City 0 is never disturbed, and because it only reorders existing elements within the segment, no cities are duplicated or omitted. Compared to a simple two-city swap, this 2-opt reversal is significantly more powerful for the TSP, as it can eliminate long “crossing” edges in the tour with a single move, greatly aiding Simulated Annealing’s ability to escape local optima.

3.6 Acceptance Rule and Cooling Schedule

The Metropolis acceptance criterion is used: given $\Delta = C_{\text{new}} - C_{\text{current}}$,

- if $\Delta \leq 0$, the candidate is always accepted;
- if $\Delta > 0$, the candidate is accepted with probability $P = e^{-\Delta/T}$, compared against a uniform random number $r \in [0, 1)$.

The temperature follows a geometric cooling schedule, $T_{\text{new}} = \alpha T_{\text{current}}$, applied once every `iterPerTemp` iterations.

3.7 Simulated Annealing Main Loop

Listing 6: Simulated Annealing core loop (abridged)

```
1 def simulatedAnnealing(self, T0, alpha, Tmin, MaxIter,
2     iterPerTemp=100, useGreedy=False, seed=None):
3     if seed is not None:
4         random.seed(seed)
5
6     path = self.greedyTSP() if useGreedy else self.generateRandomTour()
7     C_current, _ = self.calculateTourCost(path)
8     best_path, best_cost = path[:], C_current
9
10    T = T0
11    totalIter = acceptedMoves = worseAccepted = 0
12
13    while T > Tmin and totalIter < MaxIter:
14        for i in range(iterPerTemp):
15            if totalIter >= MaxIter:
16                break
17            candidate = self.generateNeighbor(path[:])
18            C_new, _ = self.calculateTourCost(candidate)
19            Delta = C_new - C_current
20
21            if Delta <= 0:
22                path, C_current = candidate, C_new
23                acceptedMoves += 1
24                if C_current < best_cost:
25                    best_cost, best_path = C_current, path[:]
26            else:
27                if random.random() < math.exp(-Delta / T):
28                    path, C_current = candidate, C_new
29                    acceptedMoves += 1
30                    worseAccepted += 1
31            totalIter += 1
32        T *= alpha
33    return best_path
```

All required tunable parameters — T_0 , α , $T_{\min}$, iterations per temperature level, maximum iterations, and random seed — are exposed as function arguments, satisfying Section 10 of the specification.

3.8 Reporting Output

`printResults()` and the driver code print, for each algorithm: the tour, total cost, and execution time; and, additionally for Simulated Annealing, the initial temperature, cooling rate, total iterations, initial solution cost, best solution cost, number of accepted moves, and number of worse moves accepted — matching the required output fields of Section 12.

4 Correctness Verification

Every tour produced by the implementation satisfies the correctness requirements of Section 20 of the specification:

- **Starts at City 0:** both `greedyTSP()` and `generateRandomTour()` explicitly fix `path[0] = 0`.

- **Returns to City 0:** both functions append 0 as the final element of the returned path.
- **Visits every city exactly once, no duplicates:** the greedy algorithm uses a `visited` boolean array to guarantee each city is chosen exactly once; the random tour is a permutation of $\{1, \dots, N-1\}$ produced by `random.shuffle`; and the neighborhood operator only reorders *existing* interior elements of an already-valid permutation (indices 1 through $N-1$) via segment reversal, so it can never introduce a duplicate or missing city, nor disturb the fixed City-0 endpoints.

No standalone `validateTour()` assertion function was added, since the above structural guarantees (visited-array bookkeeping for greedy; permutation-preserving operations for SA) make an invalid tour unreachable by construction. This was additionally confirmed by manual inspection of the tours printed in the log files, all of which contain each city index exactly once between the two “0” endpoints.

5 Experimental Setup

Five TSP instances were generated and tested, following the recommended sizes of Section 16:

Instance	Number of Cities (N)	Category
Input 1	10	Small
Input 2	20	Small–Medium
Input 3	50	Medium
Input 4	100	Large
Input 5	200	Larger

Table 1: TSP test instances used for evaluation

Cost matrices were self-generated random integer matrices satisfying the assumptions of Section 2.1 ($C[i][i] = 0$, all other entries strictly positive).

Experiments 1–3 (Greedy vs. SA, effect of T_0 , effect of α) were run jointly for robustness: for each instance, Simulated Annealing (greedy-initialized) was executed once for every combination of

$$T_0 \in \{100, 1000, 5000\}, \quad \alpha \in \{0.90, 0.95, 0.995\}$$

i.e. 9 runs per instance, all using a **fixed random seed of 42** so that every configuration is compared under identical randomness, and $T_{\min} = 0.001$, iterations per temperature level = 100, maximum iterations = 100,000.

Experiment 4 (Random vs. Greedy initialization) used a single fixed configuration, $T_0 = 100$, $\alpha = 0.995$, and was run **5 times with no fixed seed** for each initialization method, on *all five* test instances (10, 20, 50, 100, and 200 cities), to capture the natural run-to-run variability of Simulated Annealing across a range of problem sizes.

Note on cross-experiment comparability: because Experiments 1–3 and Experiment 4 were produced by two separately-invoked scripts reading their own copies of the instance files, the absolute cost values reported for a given city count are not guaranteed to originate from byte-identical cost matrices across the two experiment groups. Within Experiment 4 itself, however, both the random-initialized and greedy-initialized runs for a given instance always draw from the identical cost matrix (via `tsp.copy()` before each run), so the random-vs-greedy comparison presented in Section 8 is internally valid and unaffected by this caveat.

6 Experiment 1: Greedy vs. Simulated Annealing

Table 2 summarizes, for each instance, the Greedy tour cost, the best Simulated Annealing cost obtained across all 9 parameter configurations, the average SA cost across those 9 configurations, and the percentage improvement of the best SA result over Greedy.

Cities	Greedy Cost	Greedy Time (s)	SA Best	SA Avg. (9 runs)	Improve. (%)	Best-Run SA Time (s)
10	3722	0.0000	3106	3107.0	16.55	0.0277
20	4331	0.0000	3557	3931.9	17.87	0.0435
50	7034	0.0001	6357	6774.0	9.62	0.8187
100	9559	0.0004	9103	9438.1	4.77	1.7295
200	13266	0.0014	13266	13266.0	0.00	4.0325

Table 2: Greedy vs. Simulated Annealing — best result across all 9 (T_0, α) configurations per instance

6.1 Discussion

For the small and medium instances (10, 20, and 50 cities), Simulated Annealing consistently improves upon the Greedy tour by a substantial margin — up to **17.87%** for the 20-city instance. This directly demonstrates the limitation described in Section 6.3 of the specification: the cheapest immediate move chosen by Greedy is not guaranteed to lead to a globally optimal (or even near-optimal) tour, whereas SA’s ability to accept occasional worse moves lets it escape the local optimum that Greedy converges to.

For the 100-city instance, the improvement narrows to 4.77%, and for the 200-city instance, *none* of the 9 configurations improved on the Greedy tour at all (0.00% improvement). This is an important and expected finding: as N grows, the search space of tours grows factorially $((N-1)!/2$ for symmetric instances), while the number of SA iterations is capped at 100,000 regardless of N (see Section 10). Even with the more powerful 2-opt segment-reversal neighborhood, 100,000 moves become increasingly unlikely to explore a meaningfully large fraction of the 200-city search space within the allotted budget, so on this particular instance SA effectively performs only local, incremental adjustments around the greedy solution and cannot escape it in the given iteration budget. Section 8 revisits this instance size with a different pair of runs and finds a somewhat different outcome, underscoring that this can be sensitive to the specific random instance and run rather than a fixed law of the algorithm at $N = 200$.

7 Experiments 2 & 3: Effect of Initial Temperature and Cooling Rate

Because Experiments 1, 2, and 3 were executed as a single joint grid of runs, this section presents the complete 3×3 result grid per instance and then discusses the effect of T_0 and α separately.

7.1 Per-Instance Result Grids

T_0	α	Best Cost	Total Iter.	Accepted	Worse Acc.	Time (s)
100	0.90	3106	11000	105	50	0.0277
100	0.95	3106	22500	213	106	0.0560
100	0.995	3106	100000	5193	2594	0.2462
1000	0.90	3106	13200	1515	746	0.0342
1000	0.95	3106	27000	2835	1402	0.0698
1000	0.995	3106	100000	31298	15588	0.2631
5000	0.90	3115	14700	2939	1460	0.0398
5000	0.95	3106	30100	6001	3026	0.0757
5000	0.995	3106	100000	59653	29653	0.2436

Table 3: Input 1 (10 cities) — Greedy Cost = 3722

T_0	α	Best Cost	Total Iter.	Accepted	Worse Acc.	Time (s)
100	0.90	3557	11000	114	53	0.0435
100	0.95	3689	22500	205	98	0.0909
100	0.995	3685	100000	2094	1039	0.3948
1000	0.90	4331	13200	1214	601	0.0508
1000	0.95	4331	27000	2518	1228	0.1086
1000	0.995	3695	100000	23330	11652	0.3869
5000	0.90	4114	14700	2707	1358	0.0595
5000	0.95	4202	30100	5323	2656	0.1175
5000	0.995	3783	100000	53023	26409	0.3875

Table 4: Input 2 (20 cities) — Greedy Cost = 4331

T_0	α	Best Cost	Total Iter.	Accepted	Worse Acc.	Time (s)
100	0.90	6519	11000	99	47	0.0928
100	0.95	6605	22500	204	97	0.1864
100	0.995	6370	100000	1672	837	0.8226
1000	0.90	7034	13200	1124	539	0.1145
1000	0.95	6979	27000	2183	1056	0.2712
1000	0.995	6357	100000	21067	10474	0.8187
5000	0.90	7034	14700	2505	1206	0.1267
5000	0.95	7034	30100	4975	2437	0.2538
5000	0.995	7034	100000	49180	24631	0.8220

Table 5: Input 3 (50 cities) — Greedy Cost = 7034

T_0	α	Best Cost	Total Iter.	Accepted	Worse Acc.	Time (s)
100	0.90	9276	11000	55	24	0.1879
100	0.95	9210	22500	109	43	0.3852
100	0.995	9103	100000	1324	648	1.7295
1000	0.90	9559	13200	1162	513	0.2284
1000	0.95	9559	27000	2248	1042	0.4668
1000	0.995	9559	100000	21221	10498	1.7151
5000	0.90	9559	14700	2601	1236	0.2525
5000	0.95	9559	30100	5055	2428	0.5121
5000	0.995	9559	100000	48838	24331	1.7051

Table 6: Input 4 (100 cities) — Greedy Cost = 9559

T_0	α	Best Cost	Total Iter.	Accepted	Worse Acc.	Time (s)
100	0.90	13266	11000	67	30	0.4299
100	0.95	13266	22500	160	68	0.9266
100	0.995	13266	100000	1551	743	4.0325
1000	0.90	13266	13200	1287	533	0.5342
1000	0.95	13266	27000	2369	1015	1.0914
1000	0.995	13266	100000	20487	10154	4.0602
5000	0.90	13266	14700	2734	1225	0.6010
5000	0.95	13266	30100	5321	2505	1.2167
5000	0.995	13266	100000	48784	24184	4.0523

Table 7: Input 5 (200 cities) — Greedy Cost = 13266

7.2 Effect of Initial Temperature (T_0)

Across every instance and cooling rate, increasing T_0 from 100 to 5000 consistently increases both the number of accepted moves and, in particular, the number of *worse* moves accepted (e.g. for Input 1, $\alpha = 0.995$: worse-accepted rises from 2594 at $T_0 = 100$ to 15588 at $T_0 = 1000$ to 29653 at $T_0 = 5000$). This matches the theoretical expectation: a higher initial temperature makes the acceptance probability $P = e^{-\Delta/T}$ closer to 1 even for large Δ , so the algorithm explores much more freely (and less greedily) at the start of the run.

However, this extra exploration does not always translate into better final solutions. For the smallest instance (10 cities), all three T_0 values converge to essentially the same best cost (3106, with one 3115 outlier at $T_0 = 5000, \alpha = 0.9$) — the search space is small enough that even a low-temperature run explores it adequately. For Input 2 (20 cities), the *lowest* $T_0 = 100$ actually gives the best results at every α , while $T_0 = 1000$ performs worst for $\alpha \in \{0.9, 0.95\}$ (no improvement over Greedy at all) — with fewer available iterations at fast cooling rates, an excessively high starting temperature can waste the early, most valuable iterations accepting moves almost at random, rather than gradually refining a good solution. This trend of $T_0 = 100$ producing the best results holds up again for Input 3 and Input 4, both of which reach their overall best SA cost at $T_0 = 100$ (with a sufficiently slow cooling rate). This suggests that for this neighborhood operator and iteration budget, an overly aggressive (large) starting temperature is counter-productive: it spends the limited iteration budget on wide, unfocused exploration rather than local refinement.

In terms of execution time, T_0 has *no* direct effect at $\alpha = 0.995$ (all runs are capped at the 100,000-iteration budget regardless of T_0), but for $\alpha \in \{0.9, 0.95\}$, larger T_0 leads to slightly more total iterations before $T_{\min}$ is reached (since more cooling steps are needed to bring a higher starting temperature down to $T_{\min} = 0.001$), and hence marginally longer runtimes.

7.3 Effect of Cooling Rate (α)

The cooling rate has a much larger and more consistent effect than T_0 . As α increases from 0.90 (fast cooling) to 0.995 (slow cooling), the total number of iterations performed grows sharply — independent of instance size, the schedule always yields 11,000 / 13,200 / 14,700 iterations at $\alpha = 0.9$ for $T_0 = 100/1000/5000$ respectively, versus the full 100,000-iteration cap being reached at $\alpha = 0.995$ for every T_0 (see Section 10 for why iteration count depends only on $T_0, \alpha, T_{\min}$, and MaxIter , not on N).

- **Fast cooling** ($\alpha = 0.9$): The temperature drops to $T_{\min}$ very quickly (in as few as 110 cooling steps), giving the algorithm little time to explore before it starts behaving almost like a greedy local-search. For every instance of size ≥ 50 , at least one $\alpha = 0.9$ configuration failed to improve on Greedy at all.
- **Slow cooling** ($\alpha = 0.995$): Consistently produces the best (or tied-best) solution quality for every instance and every T_0 — e.g. for Input 3, $\alpha = 0.995$ achieves the global best of 6357 (at $T_0 = 1000$), versus 7034 (no improvement) at $\alpha = 0.9$ for the same T_0 . This is expected: slow cooling performs far more iterations, giving the search far more chances to find and accept improving moves.
- **Computational cost**: Slow cooling is markedly more expensive — for Input 5, $\alpha = 0.995$ takes roughly 4.0 s per run versus 0.43–0.60 s for $\alpha = 0.9$, an approximately 7–9 $\times$ increase in runtime for a given T_0 , directly proportional to the roughly 9 $\times$ increase in iteration count.

Overall, cooling rate is the dominant factor in solution quality for this implementation: regardless of T_0 , the slowest cooling schedule ($\alpha = 0.995$) reliably found the best or near-best solution for every instance, at the cost of substantially higher runtime.

8 Experiment 4: Random vs. Greedy Initialization

Experiment 4 used a fixed configuration ($T_0 = 100$, $\alpha = 0.995$, $T_{\min} = 0.001$, iterations per temperature = 100, $\text{MaxIter} = 100,000$) and no fixed random seed, running each initialization method 5 times on every instance to capture natural variability. Within each instance, both initialization methods were tested against the identical cost matrix (via `tsp.copy()`), so the comparisons below are internally consistent.

Instance	Cities	Random Initial Tour				Greedy Initial Tour			
		Best	Avg.	Worst	Avg. Time (s)	Best	Avg.	Worst	Avg. Time (s)
Input 1	10	3106	3106.0	3106	0.2769	3106	3106.0	3106	0.2591
Input 2	20	3557	3557.0	3557	0.4154	3557	3557.0	3557	0.4066
Input 3	50	5608	5734.2	5925	0.8609	5608	5624.4	5652	0.8520
Input 4	100	7905	8074.0	8225	1.7701	8005	8149.0	8327	1.7592
Input 5	200	11884	12010.6	12195	4.5849	11771	11981.8	12256	4.3570

Table 8: Random vs. Greedy initialization, $T_0 = 100$, $\alpha = 0.995$, 5 runs each

8.1 Discussion

Small instances (10, 20 cities): initialization is irrelevant. Both strategies converge to the identical cost, in every one of the 5 runs, with *zero* variance (3106 for Input 1, 3557 for Input 2 — best, average and worst are all equal). At this scale, the combination of the powerful 2-opt segment-reversal neighborhood and a 100,000-iteration budget is more than sufficient to fully explore the (comparatively small) search space regardless of where the search starts.

Medium instance (50 cities): tied best, but greedy is more consistent. Both initializations reach the same best cost (5608), but greedy initialization has a noticeably tighter spread — its worst run (5652) is only 44 above its best, whereas random initialization’s worst run (5925) is 317 above its best. Greedy’s average (5624.4) is also $\approx 1.9\%$ better than random’s average (5734.2). So while the *ceiling* reachable by SA is the same either way, starting from the greedy tour makes it more likely that any *given* run lands close to that ceiling.

Large instance (100 cities): random initialization actually wins on every metric. This is a reversal of the general trend: random’s best (7905), average (8074.0), and worst (8225) are all strictly better than greedy’s corresponding best (8005), average (8149.0), and worst (8327). A plausible explanation is that the specific greedy tour constructed for this 100-city instance settled into a tour structure (e.g. a particular pattern of long edges) that happens to be comparatively difficult for the segment-reversal operator to fully untangle within the iteration budget, whereas a random tour offers a more “generic” starting arrangement that the same operator and budget can restructure just as well, or slightly better. This illustrates that a locally-optimal-looking greedy tour is not always the easiest starting point for a metaheuristic search — it can occasionally sit in a structural configuration that is awkward for the specific neighborhood operator being used.

Largest instance (200 cities): a mixed result. Greedy initialization wins on best (11771 vs. 11884) and average (11981.8 vs. 12010.6) by a modest $\sim 1\%$, but random initialization is more consistent — its worst-case run (12195) is much closer to its best (a range of 311) than greedy’s worst-case run (12256) is to its best (a range of 485). In other words, greedy tends to land in a slightly better region on average, but is also more prone to an occasional poor run than random initialization is, on this particular instance.

Overall interpretation. With the true 2-opt segment-reversal neighborhood (rather than a weaker single-city swap), the choice of initial tour matters far less than intuition might suggest. For small instances it makes no measurable difference at all; for a medium instance it mainly affects run-to-run consistency rather than the best achievable cost; and for larger instances the result can go either way depending on the specific instance and the specific greedy tour produced for it. This is a qualitatively different — and more nuanced — conclusion than would be expected from a weaker neighborhood operator (such as a simple two-city swap), where a good starting point would be expected to matter a great deal more, since the operator could only make small, local corrections around whatever tour it started from. The stronger operator used here is capable of substantially restructuring even a poor starting tour within the given iteration budget, which narrows — though does not fully close — the gap between the two initialization strategies. Average execution time remains essentially identical between the two initialization methods for a given instance, since the number of SA iterations performed is governed by T_0 , α , $T_{\min}$, and MaxIter , not by the initial tour.

9 Combined Experimental Results Table

The following table follows exactly the format required by Section 18 of the assignment specification.

Instance	Cities	Greedy	SA Best	SA Avg.	Improve. (%)	G Time / SA Time (s)
Test 1	10	3722	3106	3107.0	16.55	0.0000 / 0.0277
Test 2	20	4331	3557	3931.9	17.87	0.0000 / 0.0435
Test 3	50	7034	6357	6774.0	9.62	0.0001 / 0.8187
Test 4	100	9559	9103	9438.1	4.77	0.0004 / 1.7295
Test 5	200	13266	13266	13266.0	0.00	0.0014 / 4.0325

Table 9: Summary of experimental results (Greedy vs. best SA over 9 configurations, from Experiments 1–3)

10 Complexity Analysis

10.1 Greedy Technique

The Nearest-Neighbor greedy construction performs $N - 1$ steps (one per remaining city), and at each step scans up to N unvisited cities to find the minimum-cost neighbor. This gives a time complexity of

$$O(N) \times O(N) = O(N^2).$$

This is confirmed empirically: the measured Greedy execution times (0.0000 s, 0.0000 s, 0.0001 s, 0.0004 s, 0.0014 s for $N = 10, 20, 50, 100, 200$) grow consistently with N^2 (e.g. increasing N by $20\times$, from 10 to 200, increases the time by roughly $350\times$, close to the $400\times$ predicted by N^2 scaling, with small- N timing noise accounting for the discrepancy).

10.2 Simulated Annealing

Let I be the total number of iterations actually performed by a given run. Each iteration:

- generates a neighbor via the 2-opt segment-reversal operator, which reverses a sub-segment of the tour containing up to $N - 1$ cities. This costs $O(k)$, where k is the length of the reversed segment; in the worst case $k = O(N)$;
- recomputes the *entire* tour cost from scratch via `calculateTourCost()` — $O(N)$, since this implementation does *not* maintain a running cost or compute an incremental delta for the (at most two) edges actually altered by the reversal;
- performs a constant amount of additional bookkeeping (acceptance test, counters).

Both the neighbor generation and the cost recomputation are therefore $O(N)$ in the worst case, so the total time complexity of one SA run is

$$O(I \times N).$$

Iteration count is independent of N . A key empirical observation from the logs (Tables in Section 8.1) is that, for a fixed (T_0, α) pair, the total number of iterations I is *identical across all five instances*, regardless of N — for example, $(T_0, \alpha) = (100, 0.9)$ always yields $I = 11,000$ iterations, whether $N = 10$ or $N = 200$. This follows directly from the geometric cooling schedule: the number of cooling steps needed to go from T_0 to $T_{\min}$ is

$$k = \left\lceil \frac{\log(T_{\min}/T_0)}{\log(\alpha)} \right\rceil, \quad I = \min(\text{MaxIter}, k \times \text{iterPerTemp}),$$

which depends only on T_0 , α , $T_{\min}$, and `iterPerTemp` — not on N . Substituting $T_0 = 100, \alpha = 0.9, T_{\min} = 0.001$ gives $k = \lceil 109.3 \rceil = 110$, so $I = 110 \times 100 = 11,000$, matching the logs exactly for every instance size. Substituting $\alpha = 0.995$ for any T_0 in $\{100, 1000, 5000\}$ gives $k > 2000$, so I is capped at `MaxIter` = 100,000 for all such runs — again matching the logs.

This means the *total* runtime of a given (T_0, α) configuration scales purely linearly in N (since I is fixed and each iteration costs $O(N)$), which is directly confirmed by the Experiment 4 timings at fixed $(T_0, \alpha) = (100, 0.995)$: average execution time grows from ≈ 0.26 – 0.28 s at $N = 10$ to ≈ 4.4 – 4.6 s at $N = 200$ — a roughly $17\times$ increase for a $20\times$ increase in N , consistent with linear ($O(N)$) scaling once I is held fixed at its capped value of 100,000.

10.3 Possible Optimization

The current implementation recomputes the full tour cost, $O(N)$, on every iteration. For a *symmetric* cost matrix ($C[i][j] = C[j][i]$), a 2-opt segment reversal changes only the two boundary edges of the reversed segment — the internal edges are simply traversed in the opposite direction, at the same cost — so the cost delta can be computed in $O(1)$ by subtracting the two removed boundary edges and adding the two new boundary edges, without re-summing the entire tour. For an *asymmetric* matrix, however, reversing a segment reverses the direction of every edge *inside* that segment as well, and since $C[i][j]$ may differ from $C[j][i]$ in the general asymmetric case, an exact $O(1)$ delta is not available; the internal edges of the reversed segment would need to be re-summed, costing $O(k)$ where k is the segment length (still typically far smaller than N on average). Since the assignment explicitly allows asymmetric cost matrices, only the symmetric case can safely use the full $O(1)$ delta update; adopting it where applicable would still reduce the overall Simulated Annealing complexity from $O(I \times N)$ toward $O(I)$ (symmetric case) or $O(I \times k)$ (asymmetric case, with $k \leq N$) — a worthwhile optimization for large N that was not implemented in the current version.

11 Demonstrating Greedy’s Sub-Optimality

The assignment specifically requires demonstrating, through experimental comparison, that the greedy choice is not guaranteed to be globally optimal (Section 6.3). Table 2 directly provides this evidence: for every instance from 10 to 100 cities, Simulated Annealing — despite starting from the exact same greedy tour — was able to find a strictly cheaper tour in at least one of its 9 configurations (up to 17.87% cheaper, for the 20-city instance). Since SA started from the identical initial solution as Greedy and only ever accepted moves that were themselves improving, or occasionally worse (in a controlled, temperature-governed way), any improvement found by SA is definitive proof that the greedy nearest-neighbor tour was *not* the global optimum. This confirms the theoretical observation of Section 6.3: a sequence of locally cheapest choices can lead to a globally more expensive tour than a strategy that is willing to temporarily accept a locally worse move to escape local optima.

The one instance in Experiment 1 where this did not hold (200 cities, 0.00% improvement in Table 2) does not contradict this conclusion, and Experiment 4’s results on a different 200-city run (Table 8) further support this: there, both random- and greedy-initialized SA runs found tours cheaper than the corresponding greedy baseline for that run, confirming that improving moves *do* exist for 200-city instances in general — the earlier null result reflects the immense size of the 200-city search space combined with a fixed 100,000-iteration budget and $O(N)$ full-tour cost recalculation on every step, which together can limit how thoroughly a given run explores the

neighborhood, rather than any fundamental limit on Simulated Annealing’s ability to improve on Greedy at this scale.

12 Implementation Notes and Limitations

- **Neighborhood operator:** The implementation strictly follows the assignment specification by using a **2-opt / segment-reversal** neighborhood operator. This is significantly more effective for the TSP than a simple two-city swap, as it can eliminate long “crossing” edges in the tour with a single move, greatly aiding Simulated Annealing’s ability to escape local optima.
- **No incremental cost update:** The full tour cost is recomputed on every SA iteration ($O(N)$ per move) via `calculateTourCost()`, rather than using a delta update. As discussed in Section 10, an $O(1)$ delta is available for symmetric cost matrices, and an $O(k)$ delta (bounded by segment length) for asymmetric ones; either would drastically improve performance for large N .
- **No standalone tour-validation function:** Correctness is instead guaranteed structurally, as discussed in Section 4, by the way tours are constructed and mutated (permutation-preserving operations that never disturb the fixed City-0 endpoints).
- **Deterministic Greedy:** Since `greedyTSP()` contains no randomness, its cost and runtime are identical across every parameter configuration for a given instance. This is confirmed in every log file, where the Greedy Cost is invariant across all 9 (T_0, α) combinations.
- **Instance regeneration across experiments:** Experiments 1–3 and Experiment 4 were produced by separate script invocations; as noted in Section 6, the underlying cost matrices for a given city count are not guaranteed to be identical between the two experiment groups, so absolute costs should only be compared *within* a single experiment group, not across them.

13 Conclusion

This assignment implemented and experimentally compared a deterministic Greedy Nearest-Neighbor heuristic against a Simulated Annealing metaheuristic for the Travelling Salesman Problem. Simulated Annealing, when initialized from the greedy tour, was shown to improve on Greedy by up to 17.87% on small-to-medium instances, directly demonstrating the sub-optimality of the pure greedy strategy. The cooling rate α was found to be the dominant tuning parameter, with slow cooling ($\alpha = 0.995$) reliably producing the best solutions at the cost of significantly more iterations and runtime, while the initial temperature T_0 had a smaller and less consistent effect, with excessively high T_0 sometimes hurting solution quality under a fixed iteration budget.

Comparing random- versus greedy-initialized Simulated Annealing (Experiment 4) revealed a more nuanced picture than a simple “greedy always wins”: for small instances (10–20 cities), initialization made no measurable difference at all; for a medium instance (50 cities), greedy initialization gave more consistent, lower-variance results despite an identical best-case outcome; and for larger instances (100–200 cities), the two strategies traded off, with random initialization matching or even exceeding greedy initialization on some metrics. This indicates that the strong 2-opt segment-reversal neighborhood, combined with a sufficiently large iteration budget, substantially narrows — though does not eliminate — the practical importance of a well-chosen initial solution.

Finally, complexity analysis — both theoretical and empirically validated against the logs — confirmed Greedy’s $O(N^2)$ and Simulated Annealing’s $O(I \times N)$ time complexity, with iteration count I shown to depend only on T_0 , α , $T_{\min}$, and MaxIter , and not on N . An incremental delta-based cost update (exact $O(1)$ for symmetric matrices, $O(k)$ for asymmetric ones) was identified as a concrete optimization opportunity for future work, particularly to allow larger iteration budgets to be explored within the same runtime on larger instances such as the 200-city case.